What is version control System?

A **Version Control System (VCS)** is a tool that tracks and manages changes made to source code over time. It helps developers collaborate, maintains a history of changes, allows them to restore previous versions, and prevents code conflicts during development.

Centralized vs Distributed Version Control

Centralized VCS	Distributed VCS (Git)
Single central server	Every developer has a complete copy of the repository
Internet is usually required	Most operations work offline
If the server fails, work can be affected	Local copies keep the project safe
Example: SVN	Example: Git

What is Git?

Git is a distributed version control system used to track changes in source code. It helps developers collaborate, maintain version history, create branches, merge changes, and restore previous versions whenever required.

Why do we use Git?

We use Git to track code changes, collaborate with multiple developers, maintain version history, create branches for new features, merge code safely, and restore previous versions if needed.

What is GitHub?

GitHub is a **cloud-based platform** that hosts Git repositories. It allows developers to store, manage, share, and collaborate on source code using git.

Can we use Git without GitHub?

Yes. Git works independently on your local machine. GitHub is only required when you want to store your repository online or collaborate with others.

Can we use GitHub without Git?

Not effectively. GitHub is designed to host Git repositories, so Git is typically used to manage and interact with those repositories.

Difference Between Git and GitHub

Git	GitHub
Git is a distributed version control system.	GitHub is a cloud-based platform that hosts Git repositories.
It is used to track and manage changes in source code.	It is used to store, share, and collaborate on Git repositories.
Git works on the local machine.	GitHub works on a remote server.
Git can be used without GitHub.	GitHub requires Git to manage repositories.
Example commands: git init, git add, git commit.	Used with commands like git push and git pull to interact with remote repositories.

Interview Answer

Git is a distributed version control system used to track and manage changes in source code, whereas **GitHub** is a cloud-based platform that hosts Git repositories and enables developers to store code,collaborate, share code, and manage projects.

What is a Repository?

A repository is a storage location where Git stores and manages a project's source code, commit history, branches, and other version control information.

Types of Repositories:

- **Local Repository** – The **Local Repository** stores all committed changes on the developer's local machine.
- **Remote Repository** – A **Remote Repository** is an online repository hosted on platforms like **GitHub**, where developers share and collaborate on code

What is git init?

git init is used to initialize a new Git repository in an existing project directory. It creates a hidden .git folder where Git stores all the repository information, such as version history, branches, and configuration.

git init is used to convert a normal project folder into a Git repository so that Git can start tracking the project's changes.

Command:

git init

What is git add?

git add moves changes from the Working Directory to the Staging Area, preparing them for a commit.

Simple Definition:

It tells Git:

"These changes are ready to be saved."

Commands:

git add Login.java
git add .

What is git status?

git status displays the current state of the repository, including modified, staged, and untracked files.

Simple Definition:

It shows what has changed in your project.

Command:

git status

What is git clone?

git clone is used to copy an existing remote Git repository to your local machine. It downloads the entire repository, including the project files, commit history, branches, and configuration.

Command:

git clone <repository-url>

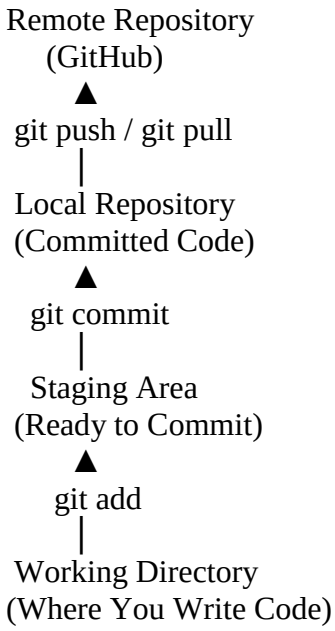
What is git commit?

git commit permanently saves the staged changes in the local repository with a meaningful message.

Command:

git commit -m "Added login functionality"

Git Workflow



What is the Working Directory?

The Working Directory is the local project folder where developers create, modify, and delete source code files before adding them to the Staging Area using git add.

What is the Staging Area?

The Staging Area is an intermediate area where changes are prepared before they are committed to the local repository.

Initially:

Working Directory

After running:

git add Login.java

The file moves to the:

Staging Area

Then you run:

git commit -m "Added login feature"

The changes are saved in the:

Local Repository

The Staging Area is an intermediate area between the Working Directory and the Local Repository where changes are prepared using git add before they are committed to the Local Repository.

What is the Local Repository?

The Local Repository stores all committed changes on the developer's local machine.

What is git push?

git push uploads committed changes from the local repository to the remote repository.

Command:

git push <remote-name> <branch-name>

git push origin main

origin → The name of the remote repository (default name for GitHub remote).
main → The branch to which the changes are pushed.

What is git pull?

git pull downloads the latest changes from the remote repository and automatically merges them into current branch of the local repository.

Command:

git pull origin main

- **git pull** → Downloads and merges the latest changes.
- **origin** → The name of the remote repository.
- **main** → The branch from which you want to pull the latest changes.

What is git fetch?

git fetch downloads the latest changes from the remote repository without merging them into the local repository.

This allows us to review the changes before merging them manually.

Command:

git fetch origin

What is a Branch?

A branch is an independent line of development that allows developers to work on new features , bug fixes or enhancements without affecting the main codebase.

After the work is completed and tested, the branch can be merged into the main branch.

Command:

git branch feature-login

Creating a Branch

git branch is used to create a new branch in Git.

Syntax:

git branch <branch-name>

Example:

git branch feature-login

Switching to a Branch

git switch or git checkout is used to switch from one branch to another.

Syntax:

git switch <branch-name>

Example:

git switch feature-login

(Older command)

git checkout feature-login

Creating and Switching to a Branch

git switch -c or git checkout -b is used to create a new branch and switch to it in a single command.

Syntax:

git switch -c <branch-name>

Example:

git switch -c feature-login

(Older command)

git checkout -b feature-login

What is Merge?

Merge is the process of combining changes from one branch into another branch.

It is used to integrate completed work into the target branch.

Command:

git merge feature-login

What is a Merge Conflict?

A Merge Conflict occurs when Git cannot automatically merge changes because two or more developers have modified the same file or the same lines of code. The conflict must be resolved manually before completing the merge.

Why does a Merge Conflict occur?

It occurs when multiple developers modify the same file or the same line of code in different branches, and Git cannot determine which version should be kept.

How do you resolve a Merge Conflict?

A merge conflict occurs when two developers modify the same file or the same lines of code. To resolve it, I first identify the conflicted file, review both developers' changes, manually choose the correct code based on the requirement, save the file, use git add to stage the resolved file, and then use git commit to complete the merge..

workflow

git merge login

|

Merge Conflict

|

Open File

|

Choose Correct Code

|

git add

|

git commit

▼

Conflict Resolved

What is a Pull Request?

A Pull Request (PR) is a request raised by a developer to merge changes from one branch into another branch. It allows team members to review the code before merging it into the main branch.

Steps to Create a Pull Request

Step 1

Create a feature branch.

```
git branch login
```

Step 2

Switch to the branch.

```
git checkout login
```

Step 3

Write the code.

Step 4

Commit the changes.

```
git add .
git commit -m "Added login automation"
```

Step 5

Push the branch.

```
git push origin login
```

Step 6

Go to **GitHub**.

Click **Create Pull Request**.

Step 7

The reviewer checks the code.

Step 8

If approved, the Pull Request is merged into main.

Why do we use a Pull Request?

We use a Pull Request to:

- Review code before merging.
- Identify bugs or issues.
- Improve code quality.
- Get approval from reviewers.
- Ensure coding standards are followed.

Who reviews a Pull Request?

Usually, the Team Lead, Senior Developer, or an assigned code reviewer reviews the Pull Request before approving and merging it.

What is git log?

git log is used to display the commit history of a Git repository, including commit ID, author, date, and commit message.

git log is a **read-only command**. It only displays history and does not modify the project.

What is git stash?

git stash is used to temporarily save uncommitted changes without committing them, allowing developers to switch tasks or branches.

What is git restore?

git restore is used to discard uncommitted changes and restore a file to its last committed state.

git restore works only for **uncommitted changes**.

Restore a single file:

git restore Login.java

Restore all modified files:

git restore .

git diff

git diff is used to compare changes between files, commits, branches, or the working directory before committing the changes.

It shows what has been added, removed, or modified in the code.

View unstaged changes (most commonly used):

git diff

View staged changes:

git diff --staged

Compare two branches:

git diff branch1 branch2

Example:

git diff main login

Compare two commits:

git diff commit_id1 commit_id2

Example:

git diff a1b2c3d e4f5g6h

Compare a specific file:

git diff Login.java

What is HEAD?

HEAD is a pointer that points to the current branch or the latest commit you are working on.

What is git reset?

Git reset is used to undo changes by moving the HEAD pointer to a previous commit.

What is git revert?

Git revert is ued to undo a committed change by creating a new commit. It does not delete the commit history.

Git reset=remove

Gitt revert=reverse

Most Important Git Commands (Interview)

Command	Purpose
git init	Initialize a repository
git clone	Copy a remote repository
git status	Check repository status
git add	Move changes to the Staging Area
git commit	Save changes to the Local Repository
git push	Upload changes to GitHub
git pull	Download and merge latest changes
git fetch	Download changes without merging
git branch	Create or list branches
git checkout	Switch branches
git switch	Switch branches (new command)
git merge	Merge one branch into another
git log	View commit history
git diff	Compare file changes
git stash	Temporarily save uncommitted changes
git restore	Discard changes in files